INFORME PROYECTO DE FISICA

Alcazar Ureña Ariel Alexander
Amelunge Guzmán Leonardo Matías
Moya Argollo Daniela Valery
Ochoa Jhonatan Williams
Ortuño Cespedes Leandro Andre
Romero Aguilar Joan Matias

April 19, 2026

1 INTRODUCCIÓN

Este informe presenta el desarrollo del proyecto de Física Teórica, incluyendo la metodología, las fórmulas matemáticas utilizadas, la implementación en Python y los resultados obtenidos.

1.1 ¿Qué es cinemática?

El estudio de la cinemática permite analizar y predecir el movimiento de los cuerpos en función del tiempo. En este proyecto se aborda un problema clásico de dos automóviles que se desplazan en la misma dirección, uno con velocidad constante y otro con aceleración constante. El objetivo es determinar la velocidad necesaria para que el automóvil A nunca rebase al automóvil B, aplicando las ecuaciones de MRU y MRUA. Además, se busca integrar la resolución matemática con herramientas computacionales como Python y documentar el proceso en LaTeX para obtener un informe académico claro y estructurado.

2 OBJETIVOS

2.1 Objetivo General

Determinar la velocidad del automóvil A que garantiza que no se acerque a menos de 10 metros del automóvil B, utilizando las ecuaciones de movimiento

rectilíneo uniforme y uniformemente acelerado.

2.2 Objetivos Específicos

- Plantear las ecuaciones de movimiento para ambos automóviles.
- Identificar el tipo de movimiento de cada automóvil (MRU y MRUA).
- Resolver matemáticamente el problema aplicando derivadas para encontrar la distancia mínima.
- Implementar el cálculo en Python para verificar los resultados.
- Documentar el proceso completo en LaTeX, incluyendo fórmulas, código y capturas de simulación.

3 METODOLOGÍA

El desarrollo del proyecto se realizó siguiendo los siguientes pasos:

1. **Planteamiento del problema:** Se definieron las condiciones iniciales de los automóviles, incluyendo velocidades, aceleración y distancia inicial.
2. **Formulación matemática:** Se establecieron las ecuaciones de movimiento para cada automóvil y la expresión de la distancia entre ellos.
3. **Análisis:** Se aplicó cálculo diferencial para determinar el instante en que la distancia entre los automóviles es mínima.
4. **Resolución:** Se impuso la condición de distancia mínima de 10 metros y se resolvió para obtener la velocidad del automóvil A.
5. **Implementación computacional:** Se programó el modelo en Python para comprobar los resultados y generar simulaciones.
6. **Documentación:** Se redactó el informe en LaTeX, integrando teoría, fórmulas, código y capturas de simulación.

4 MARCO TEÓRICO: MRU y MRUA

En este ejemplo se combinan dos tipos de movimiento:

4.1 Movimiento Rectilíneo Uniforme (MRU)

Se caracteriza por una velocidad constante y trayectoria recta. La ecuación de posición es:

$$x(t) = x_0 + v \cdot t$$

En el problema, el automóvil A se mueve con velocidad constante v_A, por lo tanto su movimiento es MRU.

4.2 Movimiento Rectilíneo Uniformemente Acelerado (MRUA)

Se caracteriza por una aceleración constante. Las ecuaciones de movimiento son:

$$v(t) = v_0 + a \cdot t$$

$$x(t) = x_0 + v_0 \cdot t + \tfrac{1}{2}at^2$$

En el problema, el automóvil B parte con velocidad inicial v_B y acelera con $a = 1\,\mathrm{m/s^2}$, por lo tanto su movimiento es MRUA.

5 HERRAMIENTAS

Las herramientas que se implemento dentro del proyecto son:

1. **Git:** Es un sistema de control de versiones que permite guardar y gestionar cambios en el código a lo largo del tiempo.

2. **Github:** Plataforma en línea que permite almacenar repositorios de código y colaborar en proyectos.

3. **Editor de código:** Software donde se escribe y ejecuta el programa (por ejemplo: Visual Studio Code).

4. **LateX:** Sistema utilizado para escribir fórmulas matemáticas y documentos científicos con formato profesional.

5. **Manim:** Biblioteca de Python utilizada para crear animaciones matemáticas y gráficas, ideal para visualizar el movimiento de los automóviles en este problema.

6. **Programación Orientada a Objetos (POO):** Paradigma de programación que organiza el código en objetos (por ejemplo, un objeto "Automóvil" con propiedades como velocidad y posición).

7. **Programación estructurada:** Forma de programar basada en el uso de funciones, secuencias y estructuras de control (if, loops), facilitando la organización del código.

8. **Overleaf:** Editor LaTeX basado en la nube, creado en 2011 por John Hammersley y John Lees-Miller. Redacción científica y técnica, especialmente artículos, tesis y presentaciones.

9. **Sympy:** Biblioteca de Python para matemáticas simbólicas (CAS: Computer Algebra System). Realizar cálculos algebraicos exactos, manipulación de expresiones y resolución de ecuaciones.

6 EJEMPLO DE CINEMATICA UNIDIMENCIONAL Num 13

El problema consiste en determinar la velocidad del automóvil A para que nunca se acerque a menos de 10 m del automóvil B.

Las ecuaciones de movimiento son:

$$x_B(t) = v_B \cdot t + \tfrac{1}{2}at^2$$

$$x_A(t) = -100 + v_A \cdot t$$

La distancia entre ellos:

$$d(t) = 100 + (v_B - v_A)t + \tfrac{1}{2}at^2$$

El mínimo ocurre cuando:

$$\frac{d}{dt}d(t) = (v_B - v_A) + at = 0$$

Evaluando en ese instante:

$$d_{\min} = 100 - \frac{(v_A - v_B)^2}{2a}$$

Imponiendo $d_{\min} = 10$:

$$v_A = v_B + \sqrt{180} \approx 32.9\,\mathrm{m/s}$$

7 FUNCIONES IMPLEMENTADAS CON SYMPY

```
t, vA = symbols('t vA')
```

“‘latex

```
xa = vA*t
```

“‘latex

```
v0b = round(v0b * (1000 / 3600),2)
print('v0b = ',v0b)
```

“‘latex

```
xB = d0 + (v0b*t) + (0.5*aB*t**2)
```

“‘latex

```
d = xB - xa
print('d =', xB,'-', xa)
```

“‘latex

```
deriv = diff(d, t)
print('0 =',deriv)
```

“‘latex

```
t_ecua = solve(deriv, t)[0]
print('t =',t_ecua)
```

“‘latex

```
xB_sus = xB.subs(t, t_ecua)
xa_sus = xa.subs(t, t_ecua)
ec_2 = xB.subs(t, t_ecua) - xa.subs(t, t_ecua)
ec_2_res = simplify(ec_2)
print(dmin,'=',ec_2_res)
```

“‘latex

```
res = Eq(dmin * 2, ec_2_res * 2)
print('vA**2 = ',solve(res, vA**2)[0])
```

“‘latex

```
vA_sol = (solve(ec_2_res - dmin, vA))
sol_redon = [round(float(s), 2) for s in vA_sol]
print(sol_redon)
```

7.1 Resultados de SymPy

$$
\begin{aligned}
v_{0B} &= 19.44 \\
d &= 0.5t^2 + 19.44t + 100 - tv_A \\
0 &= 1.0t - v_A + 19.44 \\
t &= v_A - 19.44 \\
10 &= 19.44v_A + 0.5(v_A - 19.44)^2 - 277.9136 - v_A(v_A - 19.44) \\
10 &= -0.5v_A^2 + 19.44v_A - 88.9568 \\
20 &= -1.0v_A^2 + 38.88v_A - 177.9136 \\
v_A^2 &= 38.88v_A - 197.9136 \\
v_A &= \{6.02,\ 32.86\}
\end{aligned}
$$

8 EXPLICACIÓN Y RESULRADOS IMPLEMENTANDO PYTHON

8.1 Diagrama de Flujo

En esta sección se muestra el diagrama de flujo del proyecto, implementando el proceso de las cuatro clases:
Main.py
Persecución.py
Resolución.py
Vehículo.py

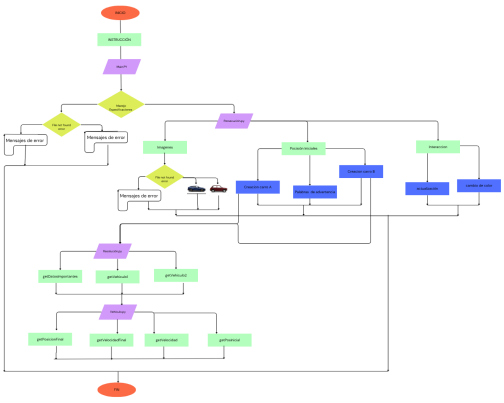

Figure 1: Fotograma completo del diagrama de flujo.

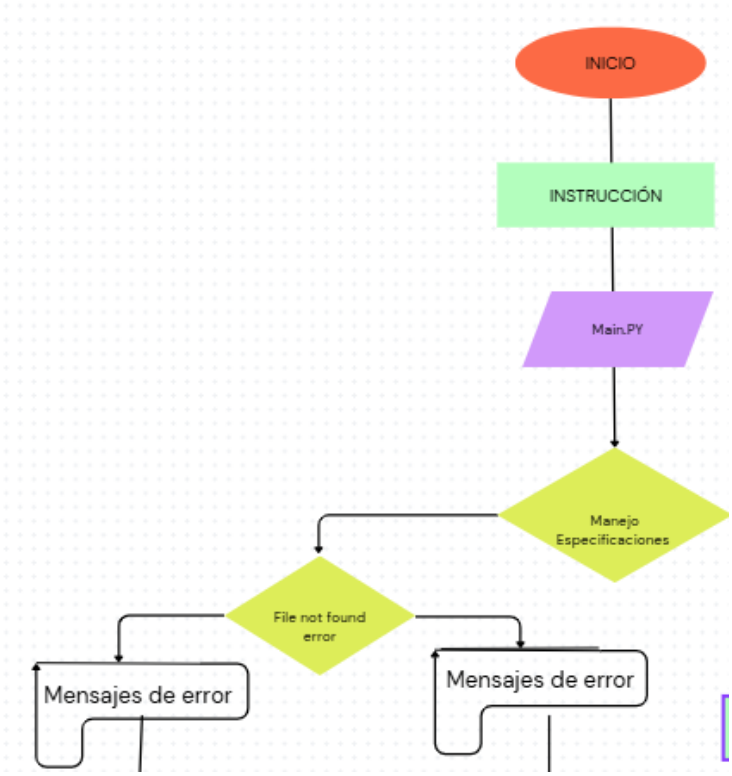

Figure 2: Fotograma del diagrama de flujo de la clase Main.py.

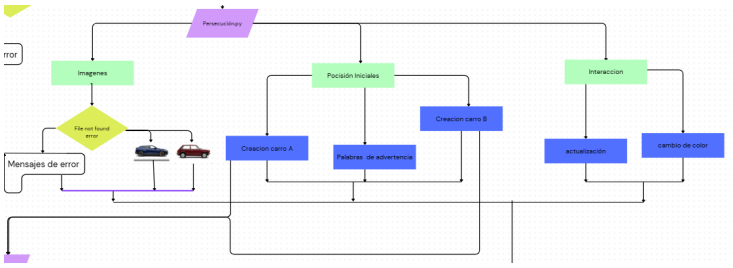

Figure 3: Fotograma del diagrama de flujo de la clase Persecución.py.

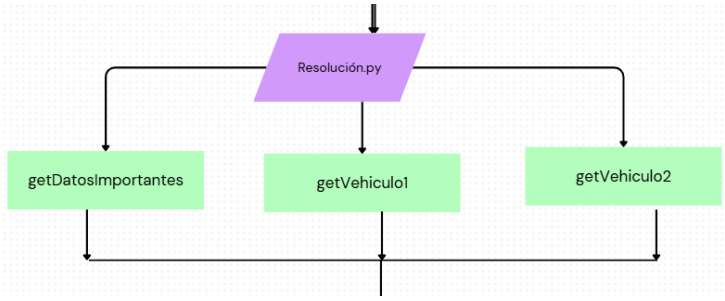

Figure 4: Fotograma del diagrama de flujo de la clase Resolución.py.

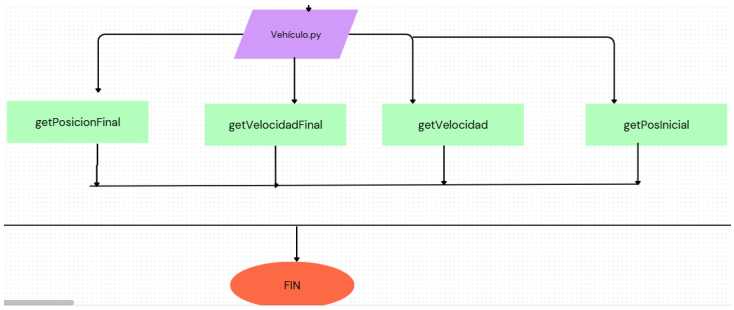

Figure 5: Fotograma del diagrama de flujo de la clase Vehículo.py.

8.2 Código Python

En esta parte del proyecto entra el verdadero reto de la programación, ya que estamos creando la simulación del el ejercicio 13 de la practica de Cinematica Unidimensional a base de código usando herramientas como Manim, Math, creando la interacción entre objetos, tratando de siempre enfocarnos en tener buenas prácticas a lo largo del desarrollo como ser: Nombres Autodescriptivos, Código legible, limpio y mantenible, respetando el mismo formato de escritura PascalCase.
Para ello el proyecto se estructuró de la siguiente manera:
Clases:

1. **Vehículo.py :** En este Archivo se creó la clase Vehiculo para crear el comportamiento y características que tiene UN solo vehículo, teniendo el enfoque de los pilares fundamentales de la Programación Orientada a Objetos(POO), como ser encapsulamiento, abstracción, teniendo métodos que acceso a las características de cada vehículo(métodos get) para el correcto comportamiento al momento de la simulación.

```python
ProyectoFisicaTeorica-main > CodigoProyecto > Vehiculo.py > Vehiculo > getPosInicial
import math
class Vehiculo:
    def __init__(self, nombre, x0, v0, aceleracion, color):
        self.nombre = nombre
        self.x0 = x0
        self.v0 = v0
        self.aceleracion = aceleracion
        self.color = color

    def getPosicionFinal(self, tiempo):
        return self.x0 + (self.v0 * tiempo) + (0.5 * self.aceleracion * math.pow(tiempo, 2))

    def getVelocidadFinal(self, tiempo):
        return self.v0 + (self.aceleracion * tiempo)

    def getVelocidad(self):
        return self.v0

    def getAceleracion(self):
        return self.aceleracion

    def getPosInicial(self):
        return self.x0
```

Figure 6: Clase Vehículo del codigo.

2. **Resolución.py :** En este archivo se implementó la interacción con la clase Vehículo, una vez construidos los dos Vehículos en la clase Resolución, se creó métodos de acceso(métodos get) para que sea más sencillo obtener datos de los Vehículos

```python
ProyectoFisicaTeorica-main > CodigoProyecto > Resolucion.py > ...
from Vehiculo import Vehiculo
from manim import BLUE, RED

class Resolucion:

    def __init__(self):
        #Resultado obtenido despues de los calculos
        self.va = 31
        self.vehiculo1 = Vehiculo("A", 0, self.va, 0, BLUE)
        self.v0b = 73 / 3.6
        self.vehiculo2 = Vehiculo("B", 100, self.v0b, 1, RED)

    def getDatosImportantes(self):
        tiempoEncuentro = (self.va - self.v0b) / self.vehiculo2.getAceleracion()
        return self.va, tiempoEncuentro, self.vehiculo2.getVelocidad()

    def getVehiculo1(self):
        return self.vehiculo1

    def getVehiculo2(self):
        return self.vehiculo2
```

Figure 7: Clase Resolución del código.

3. **Persecusión.py :** En este archivo se creó la "magia" de la simulación, se la dividió en 2 partes muy importantes, un método constructor y gestor de flujos que sería donde se invocan todos los métodos que se implementaron y una segunda parte donde esta desarrollados todos los métodos auxiliares necesarios para el proyecto, en este archivo se enfocó las buenas prácticas ya que es el archivo más grande del proyecto,

usando todas las herramientas necesarias como ser: estructuras de control condicional, estructuras de control de excepciones, construcción de la escena usando Manim,etc.
Métodos usados en todo el archivo:

- construct: Método constructor y gestor de flujos en el desarrollo de la simulación
- imagenes: Método encargado de cargar las imágenes de los autos, almacenados en un archivo aparte, usando una estructura de excepciones
- posicionesIniciales: Método encargado de almacenar los datos iniciales de los dos vehículos, estos datos los obtiene invocando a los objetos de los vehículos desde sus métodos de acceso.
- interraccion: Método encargado de la "interacción" de los dos vehículos, acá es donde se actualiza los datos de los vehículos conforme se lleva a cabo la simulación, se creó una flecha dinámica que cambia su tamaño y su color una vez se van acercando, para ello lo logra teniendo sus métodos auxiliares como ser:
- actualizacion: Método encargado de actualizar los valores de los vehículos y el color y tamaño de la flecha.
- colorFlecha: Método donde cambia el color de la flecha una vez empieza a reducir su tamaño.

ProyectoFisicaTeorica-main > CodigoProyecto > Persecucion.py > ProyectoPersecucion > interaccion

```python
from manim import *
from Resolucion import Resolucion

class ProyectoPersecucion(Scene):
    def construct(self):

        reloj = ValueTracker(0)
        res = Resolucion()

        va, tiempo, v0b = res.getDatosImportantes()

        vehiculo1 = res.getVehiculo1()
        vehiculo2 = res.getVehiculo2()

        ejes = NumberLine(x_range=[0, 380, 100], length=10).shift(DOWN * 2)

        carroA, carroB = self.imagenes()

        flecha = DoubleArrow(color=WHITE, stroke_width=1, tip_length=0.2)

        labelA, labelB = self.posicionesIniciales(carroA, carroB, ejes, vehiculo1, vehiculo2, reloj)

        self.interaccion(reloj, carroA, carroB, labelA, labelB, flecha, ejes, vehiculo1, vehiculo2)

        self.add(ejes, carroA, carroB, labelA, labelB, flecha)

        self.play(reloj.animate.set_value(tiempo), run_time=15, rate_func=linear)
        self.wait(2)
```

Figure 8: Clase Persecución del código primera parte.

```python
    def imagenes(self):
            carroA = ImageMobject("ImagenesProyecto/AutoAzul.png").scale(0.3)
            carroB = ImageMobject("ImagenesProyecto/AutoRojo.png").scale(0.3)
        except FileNotFoundError:
            print("Error: No se encontraron las imágenes PNG. Usando puntos de respaldo.")
            carroA = Dot(color=BLUE)
            carroB = Dot(color=RED)
        return carroA, carroB

    def posicionesIniciales(self, carroA, carroB, ejes, vehiculo1, vehiculo2, reloj):
        carroA.add_updater(lambda m: m.move_to(
            ejes.n2p(vehiculo1.getPosicionFinal(reloj.get_value()))
        ).shift(UP*0.7))
        carroB.add_updater(lambda m: m.move_to(
            ejes.n2p(vehiculo2.getPosicionFinal(reloj.get_value()))
        ).shift(UP*0.7))
        labelA = Text("Auto A", font_size=24).next_to(carroA, UP)
        labelB = Text("Auto B", font_size=24).next_to(carroB, UP)
        return labelA, labelB

    def interaccion(self, reloj, carroA, carroB, labelA, labelB,
                    flecha, ejes, vehiculo1, vehiculo2):

        carroA.add_updater(lambda m: m.move_to(
            ejes.n2p(min(
                vehiculo1.getPosicionFinal(reloj.get_value()),
                vehiculo2.getPosicionFinal(reloj.get_value()) - 10
            ))
        ).shift(UP*0.7))
```

Figure 9: Clase Persecución del código segunda parte.

```python

        carroB.add_updater(lambda m: m.move_to(
            ejes.n2p(vehiculo2.getPosicionFinal(reloj.get_value()))
        ).shift(UP*0.7))

        def actualizacion(flecha, dt=0):
            tiempo = reloj.get_value()
            pA = carroA.get_critical_point(RIGHT)
            pB = carroB.get_critical_point(LEFT)

            distancia = abs(vehiculo1.getPosicionFinal(tiempo) - vehiculo2.getPosicionFinal(tiempo))
            colorActual = colorCarro(distancia)

            flecha.set_color(colorActual)
            labelA.set_color(colorActual)
            labelB.set_color(colorActual)
            flecha.put_start_and_end_on(pA, pB)

        flecha.add_updater(actualizacion)

        def colorCarro(distancia):
            if distancia > 60:
                return WHITE
            elif 45 < distancia <= 60:
                return PINK
            else:
                return RED
```

Figure 10: Clase Persecución del código tercera parte.

4. **Main.py :** Se implementó este archivo por el motivo de tener un "Administrador" de las clases, con solo ejecutar esta clase funcione todo lo que viene siendo la simulación, para ello se usó herramientas poderosas e importantes como ser: Subprocess y Sys. La implementación de estos módulos permite que el proyecto cumpla con el principio de encapsulamiento, donde el usuario final interactúa con una interfaz simple (un solo script de ejecución) mientras que la complejidad técnica del renderizado matemático queda oculta y gestionada por el Administrador

del Sistema, para ello se implementó una estructura de manejo de excepciones. "La inclusión de este mecanismo de control asegura que el 'Administrador' no solo ejecute las tareas, sino que también supervise la integridad de los procesos, proporcionando un entorno de ejecución seguro y controlado para la simulación física

ProyectoFisicaTeorica-main > CodigoProyecto > Main.py > ejecutar_manim

```python
import subprocess
import sys

def ejecutar_manim():
    """
    Lanza el renderizado de Manim usando subprocess para capturar
    la salida de la terminal de Linux Mint.
    """

    comando = ["manim", "-pql", "Persecucion.py", "ProyectoPersecucion"]

    try:

        subprocess.run(comando, check=True)
    except FileNotFoundError:
        print("Error: No se encontró 'manim'. ¿Instalaste las dependencias?")
    except subprocess.CalledProcessError:
        print("Error: Hubo un problema al renderizar la escena.")

if __name__ == "__main__":
    ejecutar_manim()
```

Figure 11: Clase Principal del código

8.3 Simulación

En esta sección se muestran los resultados de la simulación paso a paso, acompañados de imágenes representativas:

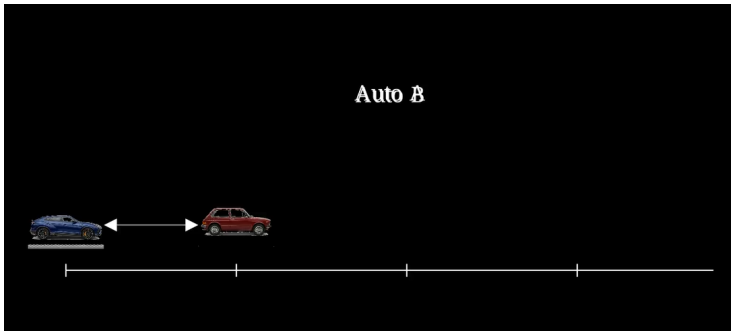

Figure 12: Primer fotograma de la simulación.

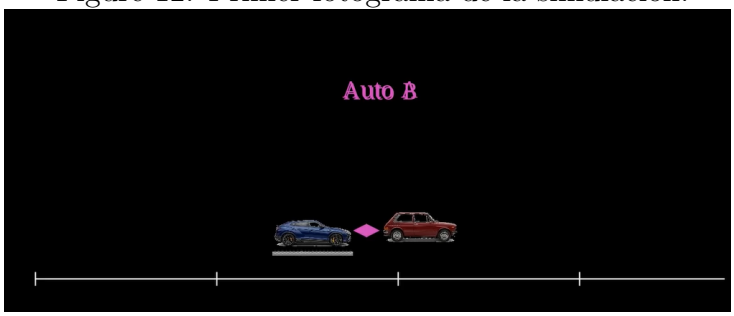

Figure 13: Segundo fotograma de la simulación.

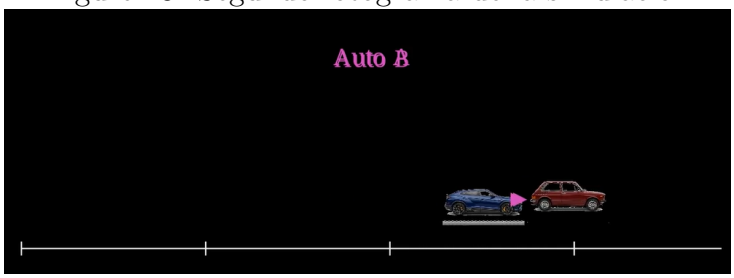

Figure 14: Tercer fotograma de la simulación.

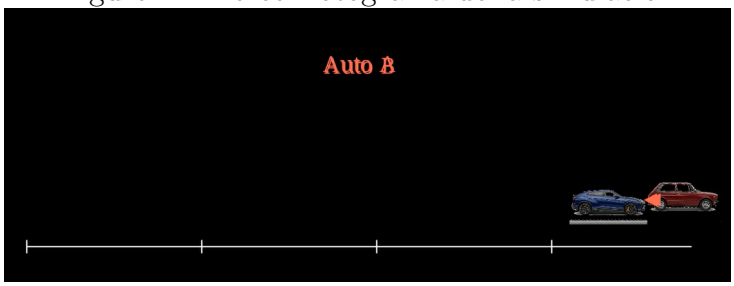

Figure 15: Cuarto fotograma de la simulación.

9 Conclusiones

Se integraron las ecuaciones de cinemática en un modelo computacional con Python, documentado en LaTeX, obteniendo un resultado consistente con la teoría.

- Se identificó correctamente el tipo de movimiento de cada automóvil: MRU para el automóvil A y MRUA para el automóvil B.
- Se aplicaron las ecuaciones de movimiento y el cálculo diferencial para determinar la condición de distancia mínima entre ambos vehículos.
- Se comprobó que la velocidad del automóvil A debe ser aproximadamente 32.9 m/s para garantizar que nunca se acerque a menos de 10 metros del automóvil B.
- La implementación en Python permitió verificar los resultados teóricos y generar simulaciones que refuerzan la comprensión del problema.
- La documentación en LaTeX aseguró una presentación clara, estructurada y académica del proceso completo, integrando teoría, fórmulas, código y capturas de simulación.

En conclusión, el proyecto demuestra la importancia de combinar fundamentos teóricos, herramientas computacionales y una adecuada documentación científica para abordar problemas de física aplicada. Esta integración no solo facilita la resolución precisa, sino que también fortalece las competencias en programación, análisis matemático y comunicación académica.